

목차

1장 바깥에서 들어오는 데이터	2
1.1 바깥에서 오는 데이터	2
1.2 신뢰할 수 없는 입력	3
1.3 형식과 의미의 구분	4
1.4 확인하지 않은 값이 퍼지는 방식	5
1.5 입력이 지나가는 자리들	6
1.6 데이터가 들어오는 자리	7
1.7 무엇까지 데이터로 볼 것인가	8
2장 스키마와 타입 경계	9
2.1 스키마의 정의	9
2.2 타입이 막아 주는 것과 못 막는 것	10
2.3 필수와 선택	11
2.4 값의 범위와 형식 규칙	12
2.5 스키마 위반을 알리는 방법	13
2.6 스키마가 바뀔 때	14
2.7 스키마로 적을 수 없는 규칙	15
3장 검증의 자리와 순서	16
3.1 어디서 검증할 것인가	16
3.2 이른 검증과 늦은 검증	17
3.3 검증 단계의 순서	18
3.4 같은 검증을 여러 번 하는 문제	19
3.5 검증과 업무 규칙의 분리	20
3.6 검증 실패를 모으는 방법	21
3.7 확인한 것을 남기는 자리	22
4장 정제와 변환	23
4.1 정제의 정의	23
4.2 정제와 검증의 차이	24
4.3 표기를 하나로 맞추는 일	25
4.4 정제 전후의 자료	26
4.5 빠진 값을 다루는 방법	27
4.6 정제가 지우는 정보	28
4.7 원본을 남길지 정하는 기준	29
5장 데이터 파이프라인의 단계	30
5.1 파이프라인의 정의	30
5.2 단계 사이의 계약	31
5.3 단계별 실패 처리	32
5.4 파이프라인 단계와 자료 상태	33
5.5 다시 돌릴 수 있는 단계	34
5.6 중간 결과를 남기는 기준	35
5.7 파이프라인 밖으로 나가는 자료	36
6장 한 자료가 지나간 길	37
6.1 사례의 자료와 목적	37
6.2 받은 그대로의 자료	38
6.3 형식 확인에서 걸린 것	39
6.4 정제에서 고친 것	40
6.5 업무 규칙에서 걸린 것	41
6.6 걸린 자료를 다시 받은 방법	42
6.7 이 길에서 남긴 기록	43
7장 검증 설계 판단	44
7.1 과도한 검증의 비용	44
7.2 어디까지 막을 것인가	45
7.3 스키마를 넓힐 때와 좁힐 때	46
7.4 검증 설계 점검 항목	47
7.5 정제 규칙을 다시 볼 때	48
7.6 파이프라인을 바꾸는 순서	49
7.7 핵심 원칙 정리	50

1.1 바깥에서 오는 데이터

시스템이 다루는 자료는 두 갈래다. 안에서 만든 자료와 바깥에서 받은 자료다. 안에서 만든 자료는 만든 규칙을 이미 알고 있지만, 바깥에서 받은 자료는 어떤 규칙으로 만들어졌는지 알 수 없다. 이 책이 다루는 것은 뒤쪽이다.

바깥이라는 말은 사람이 입력한 값만 가리키지 않는다. 다른 시스템이 보낸 자료, 파일로 받은 목록, 예전 버전이 남긴 기록이 모두 바깥이다. 만든 주체가 지금 이 코드가 아니라면 규칙이 지켜졌다는 보장이 없다는 점에서 성질이 같다.

이 구분을 처음에 해 두어야 하는 이유는 검증할 자리를 정하기 위해서다. 모든 값을 모든 자리에서 확인하면 비용이 크고, 아무 데서도 확인하지 않으면 잘못된 값이 안쪽까지 들어온다. 어디가 바깥과 안의 경계인지 정하는 일이 검증 설계의 첫 단계다.

1.2 신뢰할 수 없는 입력

입력을 신뢰하지 않는다는 말은 값이 악의적이라고 가정한다는 뜻만이 아니다. 실수로 비어 있거나, 다른 뜻으로 쓰였거나, 예전 규칙으로 만들어졌을 가능성을 모두 포함한다. 대부분의 문제는 공격이 아니라 어긋난 가정에서 온다.

신뢰 경계는 값이 확인을 거쳐 들어오는 선이다. 이 선을 지난 값은 정해진 조건을 만족한다고 안쪽 코드가 가정해도 된다. 선이 어디인지 분명하지 않으면 안쪽의 모든 함수가 각자 방어 코드를 갖게 되고, 그러고도 빠진 자리가 생긴다.

그래서 신뢰 경계는 한 군데로 모으는 편이 낫다. 들어오는 자리에서 한 번 확인하고, 그 뒤로는 확인된 값이라는 약속 위에서 처리한다. 이 약속이 지켜지는지가 이 책 전체에서 반복해 확인할 기준이다.

1.3 형식과 의미의 구분

값이 올바른가라는 물음은 두 개로 나뉜다. 생김새가 맞는가와 뜻이 맞는가다. 앞쪽은 숫자 자리에 숫자가 왔는지, 날짜가 날짜 모양인지처럼 값 하나만 보면 판단할 수 있다. 뒤쪽은 그 값이 이 업무에서 말이 되는지를 묻는다.

두 물음은 확인하는 데 필요한 것이 다르다. 생김새는 값만 있으면 되고, 뜻은 다른 값이나 저장된 기록이 함께 있어야 한다. 시작일이 날짜 모양인지는 그 값만 보면 알지만, 종료일보다 앞서는지는 두 값을 함께 보아야 안다.

이 구분을 유지하면 검증을 어디에 둘지가 정해진다. 생김새 확인은 들어오는 자리에서 한 번에 끝낼 수 있고, 뜻의 확인은 필요한 자료가 모이는 자리에서 해야 한다. 둘을 한자리에서 하려 하면 들어오는 자리가 업무 규칙을 알아야 하는 상태가 된다.

1.4 확인하지 않은 값이 퍼지는 방식

확인하지 않은 값은 들어온 자리에서 멈추지 않는다. 계산에 쓰이고, 저장되고, 저장된 값이 다시 다른 계산에 쓰인다. 문제가 드러날 때쯤이면 그 값은 여러 자리에 사본으로 남아 있고, 어느 것이 원인이고 어느 것이 결과인지 가리기 어렵다.

더 나쁜 경우는 값이 조용히 다른 뜻으로 해석되는 것이다. 비어 있는 값이 영으로 읽히거나, 단위가 다른 숫자가 그대로 더해지는 일이 여기 든다. 이때는 오류가 나지 않으므로 잘못이 결과에만 남는다.

그래서 값을 들어오는 자리에서 막는 것은 편의가 아니라 원인을 찾을 수 있게 만드는 일이다. 막힌 값은 어디서 왜 막혔는지 한 자리에 기록되지만, 통과한 값은 어디까지 갔는지 되짚어야 한다. 확인은 잘못을 없애는 일이 아니라 잘못이 머무는 범위를 좁히는 일이다.

1.5 입력이 지나는 자리들

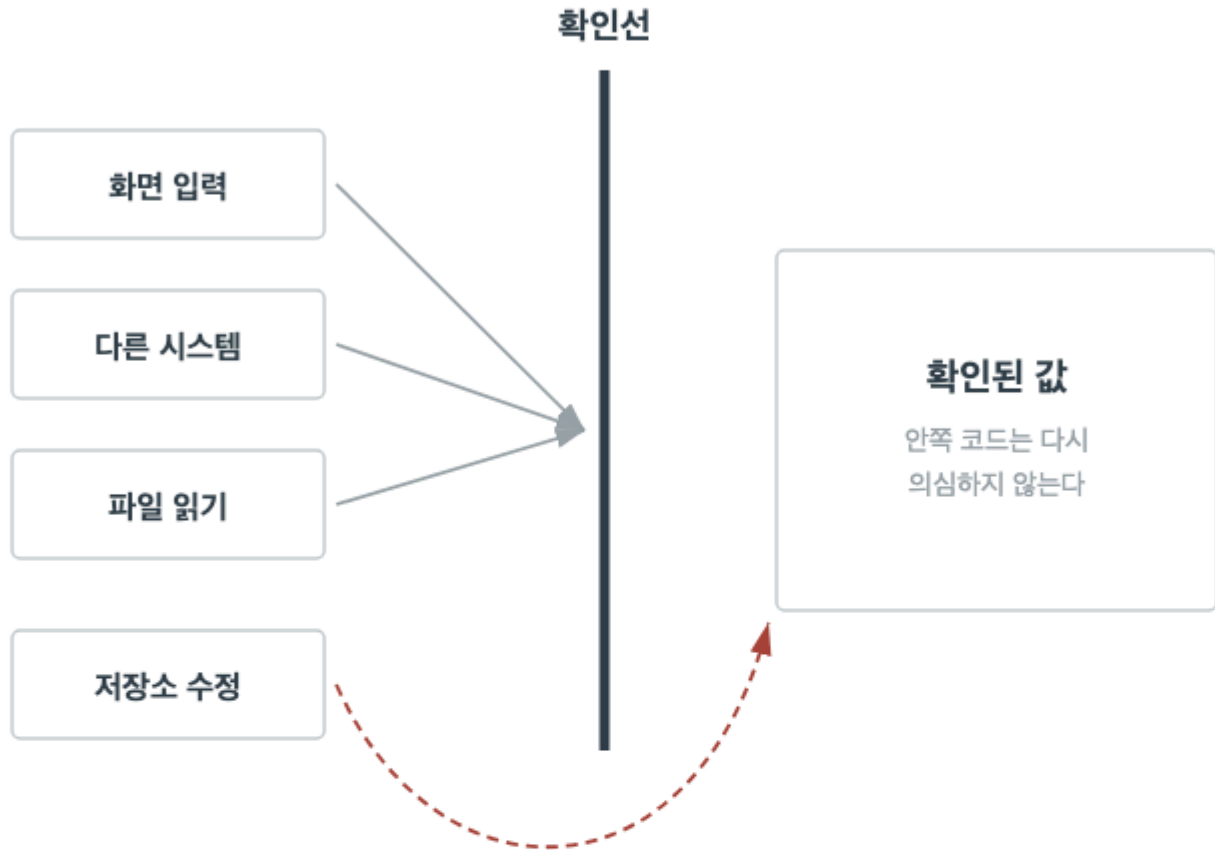
자료가 들어오는 통로는 하나가 아니다. 사용자가 화면에서 넣는 값, 다른 시스템이 보내는 요청, 정해진 시각에 읽어 오는 파일, 사람이 직접 고치는 저장소가 모두 통로다. 통로마다 양과 속도와 고칠 수 있는 여지가 다르다.

통로를 모두 적어 보면 검증이 빠진 자리가 드러난다. 대개 화면으로 들어오는 값은 꼼꼼히 확인하면서 파일이나 다른 시스템에서 오는 자료는 그대로 받아들인다. 뒤쪽이 한 번에 들어오는 양이 훨씬 많은데도 그렇다.

통로마다 검증 규칙을 따로 만들 필요는 없다. 오히려 같은 규칙을 통로마다 다시 적는 것이 어긋남의 원인이 된다. 규칙은 한 자리에 두고 통로마다 그 규칙을 부르는 편이 낫다. 통로를 세는 일은 규칙을 늘리기 위해서가 아니라 빠진 자리를 찾기 위해서다.

데이터가 들어오는 자리

통로는 여럿이고 확인선은 하나다



이 통로는 확인선을 지나지 않는다

빠진 통로는 확인선 밖에서 들어온다

1.7 무엇까지 데이터로 볼 것인가

검증 대상을 정할 때 값만 생각하기 쉽지만, 자료에는 값 말고도 확인할 것이 있다. 항목의 수, 목록의 길이, 파일의 크기, 같은 자료가 두 번 들어왔는지가 모두 대상이다. 값 하나하나를 멀쩡한데 전체가 말이 되지 않는 경우가 여기서 걸린다.

반대로 검증 대상에서 빼야 할 것도 있다. 이 시스템이 뜻을 해석하지 않고 그대로 전달만 하는 자료가 그렇다. 뜻을 모르는 자료에 규칙을 걸면 바깥 규칙이 바뀔 때마다 이 시스템이 함께 바뀐다.

판단 기준은 이 시스템이 그 자료로 무엇을 하느냐다. 값을 읽어 판단이나 계산에 쓴다면 검증 대상이고, 받아서 그대로 넘기기만 한다면 형태만 확인하고 뜻은 묻지 않는다. 이 기준이 서 있어야 검증 규칙이 필요 이상으로 늘어나지 않는다.

2.1 스키마의 정의

스키마는 자료가 어떤 항목으로 이루어지고 각 항목이 어떤 값을 가질 수 있는지 적어 둔 약속이다. 항목의 이름, 값의 종류, 있어야 하는지 없어도 되는지, 여러 개일 수 있는지가 여기 들어간다. 자료를 주고받는 양쪽이 같은 스키마를 보고 있어야 대화가 성립한다.

스키마의 값은 두 가지다. 하나는 확인을 자동으로 만들 수 있다는 것이다. 규칙이 적혀 있으면 받은 자료가 그 규칙에 맞는지 기계가 검사한다. 다른 하나는 합의의 기록이라는 것이다. 무엇을 주고받기로 했는지가 코드 밖에 남는다.

스키마는 자료의 뜻까지 담지는 못한다. 항목이 무엇을 가리키는지, 어떤 조건에서 이 자료를 보내는지는 스키마 밖에 있다. 그래서 스키마를 두는 일은 확인의 시작이지 끝이 아니다. 이 장은 스키마가 무엇까지 막아 주는지를 경계로 삼아 설명한다.

2.2 타입이 막아 주는 것과 못 막는 것

값의 종류를 정해 두면 그 종류에 맞지 않는 값이 들어오는 일은 막힌다. 숫자 자리에 글자가 오면 받아들이는 자리에서 걸린다. 이것만으로도 안쪽 코드는 값의 종류를 매번 확인하지 않아도 된다.

그러나 같은 종류 안에서 잘못된 값은 막지 못한다. 나이 자리에 음수가 오거나 수량 자리에 터무니없이 큰 수가 와도 종류는 맞다. 더 혼한 문제는 뜻이 다른 두 값이 같은 종류라서 서로 바뀌어 들어가는 것이다. 사용자 번호 자리에 주문 번호를 넣어도 둘 다 숫자면 걸리지 않는다.

그래서 종류는 첫 번째 그물이지 유일한 그물이 아니다. 종류를 좁게 잡을수록 걸리는 것이 늘지만, 좁힌 만큼 자료를 다루는 코드가 늘어난다. 어디까지 종류로 막고 어디부터 규칙으로 막을지가 이 장의 판단이다.

2.3 필수와 선택

어떤 항목이 반드시 있어야 하고 어떤 항목이 없어도 되는지는 스키마에서 가장 자주 바뀌는 부분이다. 필수로 잡으면 보내는 쪽이 늘 채워야 하고, 선택으로 잡으면 받는 쪽이 없는 경우를 늘 다뤄야 한다. 어느 쪽이든 비용은 사라지지 않고 옮겨 갈 뿐이다.

판단 기준은 그 항목 없이 처리를 끝낼 수 있는가다. 없으면 처리가 불가능한 항목은 필수이고, 없어도 결과를 낼 수 있는 항목은 선택이다. 이 기준으로 보면 '거의 항상 있는 항목'은 선택이다. 거의라는 말이 붙는 순간 없는 경우를 다뤄야 한다.

선택 항목에는 없을 때의 뜻을 함께 정해야 한다. 값이 없다는 것과 값이 비어 있다는 것과 값이 영이라는 것은 서로 다른 상태다. 셋을 같은 것으로 다루면 나중에 구분할 수 없고, 구분하려면 저장된 자료를 되짚어야 한다.

2.4 값의 범위와 형식 규칙

값의 종류 위에 얹는 조건이 값 규칙이다. 최소와 최대, 자릿수, 정해진 목록 가운데 하나, 정해진 형태를 따르는 문자열이 여기 든다. 이런 조건은 대개 업무에서 온다. 수량이 하나 이상이어야 한다는 규칙은 자료의 성질이 아니라 그 업무의 성질이다.

값 규칙을 적을 때 조심할 것은 지금의 자료를 그대로 옮기지 않는 것이다. 지금 들어오는 값이 모두 다섯 자리라고 해서 다섯 자리를 규칙으로 만들면, 자릿수가 늘어날 때 정상 자료가 막힌다. 규칙은 관찰이 아니라 약속에서 와야 한다.

규칙이 여럿 걸리는 항목은 순서도 정해야 한다. 비어 있는지 먼저 보고, 종류를 보고, 범위를 보는 차례가 자연스럽다. 순서를 정하지 않으면 비어 있는 값에 범위 검사를 걸어 뜻 없는 실패가 나온다.

2.5 스키마 위반을 알리는 방법

스키마에 맞지 않는 자료를 받았을 때 돌려줄 것은 실패했다는 사실만이 아니다. 어느 항목이, 어떤 조건에 어긋났는지가 함께 있어야 보내는 쪽이 고칠 수 있다. 항목 이름 없이 형식이 잘못되었다고만 알리면 보내는 쪽은 전체를 다시 살펴야 한다.

그렇다고 내부 사정을 그대로 내보내면 안 된다. 저장 구조나 내부 항목 이름을 담은 오류 문구는 바깥이 알 필요가 없고, 나중에 내부를 고칠 때 함께 바뀌어야 하는 짐이 된다. 알릴 것은 약속된 항목의 이름과 어긋난 조건이다.

형태를 정해 두는 것도 중요하다. 위반 하나를 어떻게 적을지 정해 두면 여러 위반을 목록으로 담을 수 있고, 받는 쪽이 기계적으로 처리할 수 있다. 문장으로만 적힌 오류는 사람이 읽기에는 좋지만 자동으로 다루기 어렵다.

2.6 스키마가 바뀔 때

스키마는 한 번 정하면 끝나지 않는다. 항목이 늘고, 뜻이 갈라지고, 쓰지 않는 항목이 남는다. 문제는 이미 예전 스키마로 만들어진 자료가 저장되어 있고 예전 스키마로 보내는 쪽도 남아 있다는 것이다.

안전한 변경과 위험한 변경은 방향으로 갈린다. 선택 항목을 더하는 것은 예전 자료도 그대로 통과하므로 안전하고, 필수 항목을 더하거나 값의 범위를 좁히는 것은 예전 자료를 막으므로 위험하다. 항목의 뜻만 조용히 바꾸는 것은 가장 위험하다. 아무것도 걸리지 않은 채 해석만 달라진다.

위험한 변경이 필요하면 한 번에 바꾸지 않고 단계를 나눈다. 새 항목을 선택으로 더해 두 스키마를 함께 받고, 보내는 쪽이 모두 옮긴 것을 확인한 뒤 예전 항목을 지운다. 순서를 지키면 자료를 잃지 않고 약속을 옮길 수 있다.

2.7 스키마로 적을 수 없는 규칙

스키마로 적을 수 있는 것은 자료 자체를 보고 판단할 수 있는 조건까지다. 두 항목의 관계, 저장된 기록과의 대조, 지금 시각에 따라 달라지는 조건은 스키마 밖에 있다. 종료일이 시작일보다 뒤인지는 적을 수 있는 스키마도 있지만, 그 주문이 이미 처리되었는지는 어느 스키마로도 적을 수 없다.

이런 조건을 스키마에 억지로 넣으려 하면 스키마가 업무 규칙을 담기 시작한다. 그러면 스키마가 업무만큼 자주 바뀌고, 주고받는 양쪽이 그 변경을 함께 감당해야 한다. 스키마의 값이던 안정성이 사라진다.

그래서 검증은 두 층으로 나뉜다. 자료만 보고 판단하는 층과 업무 상태를 함께 보는 층이다. 앞 층은 들어오는 자리에서 스키마로 하고, 뒤 층은 처리 안쪽에서 업무 규칙으로 한다. 다음 장은 이 두 층을 어디에 둘지를 다룬다.

3.1 어디서 검증할 것인가

검증을 어디서 할지는 무엇을 검증할지만큼 중요하다. 같은 규칙이라도 들어오는 자리에서 걸면 잘못된 자료가 안으로 들어오지 않고, 쓰는 자리에서 걸면 이미 저장된 뒤에 걸린다. 앞쪽은 막는 일이고 뒤쪽은 찾는 일이다.

기본 배치는 두 층이다. 자료의 생김새는 들어오는 자리에서 한 번에 확인하고, 업무 상태를 함께 보아야 하는 조건은 처리 안쪽에서 확인한다. 이렇게 나누면 들어오는 자리가 업무를 몰라도 되고, 처리 안쪽은 생김새를 다시 확인하지 않아도 된다.

자리를 정할 때 함께 정해야 할 것은 통과한 값의 지위다. 확인선을 지난 값이 확인된 값으로 다뤄진다는 약속이 없으면 안쪽 코드는 여전히 값을 의심하고, 검증 자리를 나눈 뜻이 사라진다. 검증 설계는 규칙의 목록이 아니라 이 약속의 배치다.

3.2 이른 검증과 늦은 검증

검증을 앞당기면 잘못된 자료가 안쪽으로 들어오지 않고, 실패한 원인이 들어온 자리 가까이에 남는다. 되짚을 거리가 짧아지므로 고치는 시간이 준다. 대부분의 형식 검증은 앞당기는 편이 낫다.

그러나 모든 검증을 앞당길 수는 없다. 판단에 필요한 자료가 아직 모이지 않았거나, 지금 확인해도 처리하는 시점에 상태가 달라질 수 있는 조건이 있다. 재고가 남아 있는지를 받는 자리에서 확인해도 처리하는 순간에는 달라져 있을 수 있다.

그래서 앞당길 수 있는 것과 미뤄야 하는 것을 가르는 기준은 시간에 따라 답이 달라지는가다. 값 자체로 답이 정해지는 조건은 앞으로 당기고, 상태에 따라 답이 달라지는 조건은 실제로 그 상태를 바꾸는 자리에서 확인한다.

검증 단계의 순서

앞 단계를 지나야 다음 조건을 물을 수 있다



앞 세 단계는 값만 보고 판단한다

3.4 같은 검증을 여러 번 하는 문제

같은 조건을 여러 자리에서 확인하는 것은 안전해 보인다. 실제로는 세 가지 비용이 생긴다. 첫째, 규칙이 바뀌면 여러 자리를 함께 고쳐야 하고 한 자리를 빠뜨리면 자리마다 다른 규칙이 된다. 둘째, 어느 자리에서 걸렸는지에 따라 다른 실패가 나가 보내는 쪽이 혼란스럽다.

셋째가 가장 다루기 어렵다. 확인이 여러 자리에 있으면 어느 자리가 진짜 경계인지 알 수 없어진다. 새 통로가 생겼을 때 어떤 확인을 붙여야 하는지 판단할 근거가 사라지고, 결국 모든 확인을 다시 붙이게 된다.

겹친 확인을 정리할 때는 지우기 전에 자리를 정한다. 이 조건을 책임지는 자리를 하나 고르고, 그 자리가 확인한다는 사실을 다른 자리에서 의지해도 된다고 적어 둔다. 그 뒤에 나머지를 지운다. 순서를 뒤집으면 확인이 사라진 채로 남는다.

3.5 검증과 업무 규칙의 분리

형식 확인과 업무 판단을 한 함수에 담으면 둘 다 다루기 어려워진다. 형식 확인은 자주 바뀌지 않지만 업무 판단은 자주 바뀌고, 형식 확인은 자료만 있으면 되지만 업무 판단은 저장된 기록이 필요하다. 바뀌는 속도와 필요한 것이 다른 둘을 묶으면 확인 하나를 고칠 때마다 다른 쪽까지 함께 준비해야 한다.

나눈 뒤에는 각각이 무엇을 책임지는지 적어 둔다. 형식 층은 자료가 약속된 모양인지까지 책임지고, 업무 층은 그 모양의 자료가 지금 상황에서 받아들일 수 있는지를 책임진다. 두 층 사이의 약속은 확인된 자료의 형태다.

나누면 확인을 따로 확인하기도 쉬워진다. 형식 층은 자료만 주면 결과를 볼 수 있고, 업무 층은 상태를 만들어 두고 결과를 볼 수 있다. 섞여 있으면 형식 하나를 확인하려 해도 상태를 모두 갖춰야 한다.

3.6 검증 실패를 모으는 방법

확인이 여럿일 때 처음 어긋난 자리에서 멈출지, 끝까지 보고 모두 모을지 정해야 한다. 멈추는 방식은 단순하고 빠르지만 보내는 쪽이 고쳐서 다시 보낼 때마다 다음 잘못이 하나씩 드러난다. 모으는 방식은 한 번에 알려 주지만 앞 조건이 어긋난 뒤의 확인이 뜻 없는 실패를 낼 수 있다.

실용적인 절충은 단계 안에서는 모으고 단계 사이에서는 멈추는 것이다. 항목마다 독립적인 형식 확인은 모두 모아 한 번에 알리고, 형식 확인이 실패하면 업무 확인으로 넘어가지 않는다. 뜻 없는 실패가 섞이지 않으면서도 고치는 횟수가 준다.

모을 때는 순서도 정해 두는 편이 좋다. 자료에 적힌 순서대로 모으면 보내는 쪽이 목록과 자료를 나란히 놓고 고칠 수 있다. 순서가 매번 달라지면 같은 자료를 두 번 보냈을 때 결과를 견주기 어렵다.

3.7 확인한 것을 남기는 자리

검증이 한 일은 통과와 실패로만 남지 않는다. 어떤 자료가 언제 어떤 규칙에 걸렸는지가 쌓이면 규칙 자체를 다시 볼 근거가 된다. 특정 규칙에서만 실패가 몰린다면 그 규칙이 실제 자료와 맞지 않을 수 있다.

남길 때 조심할 것은 자료 자체를 그대로 남기지 않는 것이다. 걸린 자료를 통째로 남기면 확인하지 않은 값이 다른 자리에 사본으로 남는 셈이고, 담긴 내용에 따라 남기면 안 되는 값이 섞일 수 있다. 남길 것은 어떤 항목이 어떤 규칙에 걸렸다는 사실이다.

기록의 쓸모는 규칙을 고칠 때 드러난다. 규칙을 좁히거나 넓히기로 했을 때, 그 규칙이 지금까지 무엇을 막아 왔는지 알면 영향 범위를 짐작할 수 있다. 기록이 없으면 규칙 변경은 늘 짐작으로 한다.

4.1 정제의 정의

정제는 받은 자료를 안에서 다루기 좋은 모양으로 고치는 일이다. 앞뒤 공백을 없애고, 표기를 하나로 맞추고, 단위를 통일하고, 빈 값을 정해진 표시로 바꾸는 일이 여기 든다. 자료의 뜻은 그대로 두고 모양만 바꾸는 것이 정제의 기본이다.

정제가 필요한 이유는 바깥의 표기가 하나가 아니기 때문이다. 같은 전화번호를 붙임표를 넣어 쓰는 곳과 빼고 쓰는 곳이 있고, 같은 날짜를 다른 차례로 적는 곳이 있다. 안에서 이 모두를 그대로 다루면 비교와 검색이 매번 달라진다.

정제는 검증 앞에 오기도 하고 뒤에 오기도 한다. 표기를 맞춰야 규칙을 걸 수 있는 항목은 먼저 정제하고, 원래 모양이 규칙의 대상인 항목은 확인한 뒤에 고친다. 순서를 정하지 않으면 정제가 잘못된 값을 그럴듯한 값으로 바꿔 검증을 통과시킨다.

4.2 정제와 검증의 차이

정제와 검증은 자료를 다루는 방향이 다르다. 검증은 조건에 맞지 않으면 막고, 정제는 조건에 맞게 고친다. 같은 상황에서 어느 쪽을 고를지가 이 장의 판단이다.

고쳐도 되는 것은 뜻이 하나로 정해지는 경우다. 앞뒤 공백은 없애도 뜻이 달라지지 않고, 대문자와 소문자의 구분이 뜻과 무관한 항목은 하나로 맞춰도 된다. 반대로 고치면 안 되는 것은 뜻이 여럿으로 갈리는 경우다. 자릿수가 모자란 번호를 앞에 영을 붙여 채우는 일은 다른 번호를 만들 수 있다.

판단이 서지 않을 때는 막는 쪽이 안전하다. 막으면 보내는 쪽이 고쳐 다시 보내지만, 잘못 고치면 아무도 모르는 채로 다른 자료가 된다. 정제는 편의를 위한 것이고 그 편의가 뜻을 바꾸는 순간 손해가 이익보다 크다.

4.3 표기를 하나로 맞추는 일

표기를 맞추는 때는 무엇을 기준으로 삼을지부터 정한다. 안에서 쓸 하나의 모양을 정하고 들어오는 모든 표기를 그 모양으로 옮긴다. 기준이 없으면 통일한다면서 자리마다 다른 모양이 남는다.

기준을 고를 때는 되돌릴 수 있는 쪽을 고른다. 붙임표를 뺀 번호에서 붙임표를 다시 넣는 일은 규칙만 알면 되지만, 반올림한 값에서 원래 값을 되찾을 수는 없다. 되돌릴 수 없는 변환은 정제가 아니라 계산이므로 정제 단계에서 하지 않는다.

옮기는 규칙은 자료마다 한 자리에 모아 둔다. 같은 항목의 표기 규칙이 여러 자리에 흩어져 있으면 한 자리만 고쳐진 채로 남아 같은 자료가 통로마다 다르게 저장된다. 규칙이 한 자리에 있으면 표기가 바뀌었을 때 어디를 고칠지 묻지 않아도 된다.

정제 전후의 자료

뜻이 그대로인 것만 고친다



고친 자리는 표시로 남긴다

4.5 빠진 값을 다루는 방법

값이 없는 자리를 만나면 세 가지 선택이 있다. 그대로 없음으로 두기, 정해진 기본값으로 채우기, 자료 전체를 막기다. 어느 쪽을 고르느냐에 따라 뒤의 처리가 완전히 달라진다.

기본값으로 채우는 선택은 편하지만 위험하다. 채운 순간 그 값이 원래 있었는지 채워 넣은 것인지 구분할 수 없다. 나중에 그 항목으로 통계를 내거나 조건을 걸 때 채워 넣은 값이 실제 값처럼 섞인다. 채우기로 했다면 채웠다는 사실을 함께 남겨야 한다.

없음으로 두는 선택은 뒤의 처리마다 없는 경우를 다루게 만든다. 대신 자료가 사실대로 남는다. 이 책은 뒤쪽을 기본으로 삼는다. 사실대로 남긴 자료는 나중에 채울 수 있지만, 채워 버린 자료는 되돌릴 수 없다.

4.6 정제가 지우는 정보

정제는 자료를 다루기 쉽게 만들지만 그 과정에서 정보가 사라진다. 앞뒤 공백을 없애면 보내는 쪽이 어떻게 적었는지가 사라지고, 표기를 맞추면 어느 통로에서 왔는지 짐작할 단서가 사라진다. 대개는 필요 없는 정보지만 문제를 찾을 때는 그렇지 않다.

특히 어려운 경우는 정제가 잘못을 감추는 것이다. 뜻이 어긋난 값이 정제를 거치며 그럴듯한 모양이 되면 검증을 통과하고, 통과한 뒤에는 원래 어떤 모양이었는지 알 수 없다. 이런 자료는 결과가 이상해진 뒤에야 드러난다.

그래서 정제는 무엇을 고쳤는지 남기는 것을 함께 정한다. 고친 항목의 이름과 규칙을 남겨 두면 나중에 되짚을 수 있다. 고친 값 자체를 남길지는 다음 절에서 따로 다룬다. 정제의 값은 편의이고, 그 편의에 되짚을 길을 붙여야 안전해진다.

4.7 원본을 남길지 정하는 기준

받은 자료를 그대로 남길지는 세 가지를 보고 정한다. 되짚을 일이 얼마나 잦은지, 자료의 양이 얼마나 되는지, 그 자료에 남기면 안 되는 값이 섞여 있는지도. 셋 다 답이 다르면 결론도 달라진다.

되짚을 일이 잦은 자료는 남기는 편이 낫다. 바깥에서 오는 자료는 보내는 쪽과 뜻이 어긋나는 일이 잦고, 그때 받은 그대로가 없으면 어느 쪽의 잘못인지 가릴 수 없다. 반대로 양이 크고 되짚을 일이 드문 자료는 정제 결과만 남기고 원본은 짧은 기간만 둔다.

남기면 안 되는 값이 섞인 자료는 남기지 않는 것이 기본이다. 이때는 원본 대신 어떤 항목이 어떤 규칙으로 고쳐졌는지만 남긴다. 남길 것과 남기지 않을 것을 미리 정해 두면 문제가 생긴 뒤에 판단하지 않아도 된다.

5.1 파이프라인의 정의

파이프라인은 자료가 정해진 단계를 차례로 지나며 처리되는 구조다. 받기, 확인하기, 고치기, 저장하기처럼 각 단계가 앞 단계의 결과를 받아 다음 단계로 넘긴다. 한 단계가 하는 일이 정해져 있고 단계 사이에 오가는 것이 정해져 있다는 점이 파이프라인의 핵심이다.

이 구조를 쓰는 이유는 자료의 상태를 단계로 말할 수 있어서다. 어떤 자료가 지금 어디까지 왔는지, 어느 단계에서 멈췄는지가 분명해진다. 한 덩어리 처리에서는 실패한 자료가 어디까지 처리되었는지 알 수 없어 다시 돌리기도 어렵다.

단계를 나눌 때 기준은 앞 장과 같다. 함께 바뀌는 일을 한 단계에 두고, 단계 사이에 오가는 것을 적게 유지한다. 단계가 늘어날수록 상태는 자세해지지만 단계 사이의 약속이 함께 늘어난다.

5.2 단계 사이의 계약

단계 사이에는 계약이 있다. 앞 단계가 넘기는 자료가 어떤 조건을 만족하는지, 뒤 단계가 그 조건을 다시 확인해도 되는지가 계약의 내용이다. 이 계약이 적혀 있지 않으면 뒤 단계는 자료를 다시 의심하고, 확인이 단계마다 겹친다.

계약을 적는 실용적인 방법은 단계마다 통과한 자료의 상태에 이름을 붙이는 것이다. 받은 그대로, 형식을 확인한 자료, 정제를 마친 자료, 업무 확인까지 마친 자료처럼 나누면 어느 단계가 무엇을 보장하는지가 이름으로 드러난다.

이름을 붙여 두면 단계를 바꾸기도 쉬워진다. 새 단계를 넣을 때 그 단계가 받는 상태와 내보내는 상태만 맞추면 되고, 앞뒤 단계를 열어 보지 않아도 된다. 단계 사이의 계약은 파이프라인에서 인터페이스가 하는 일과 같다.

5.3 단계별 실패 처리

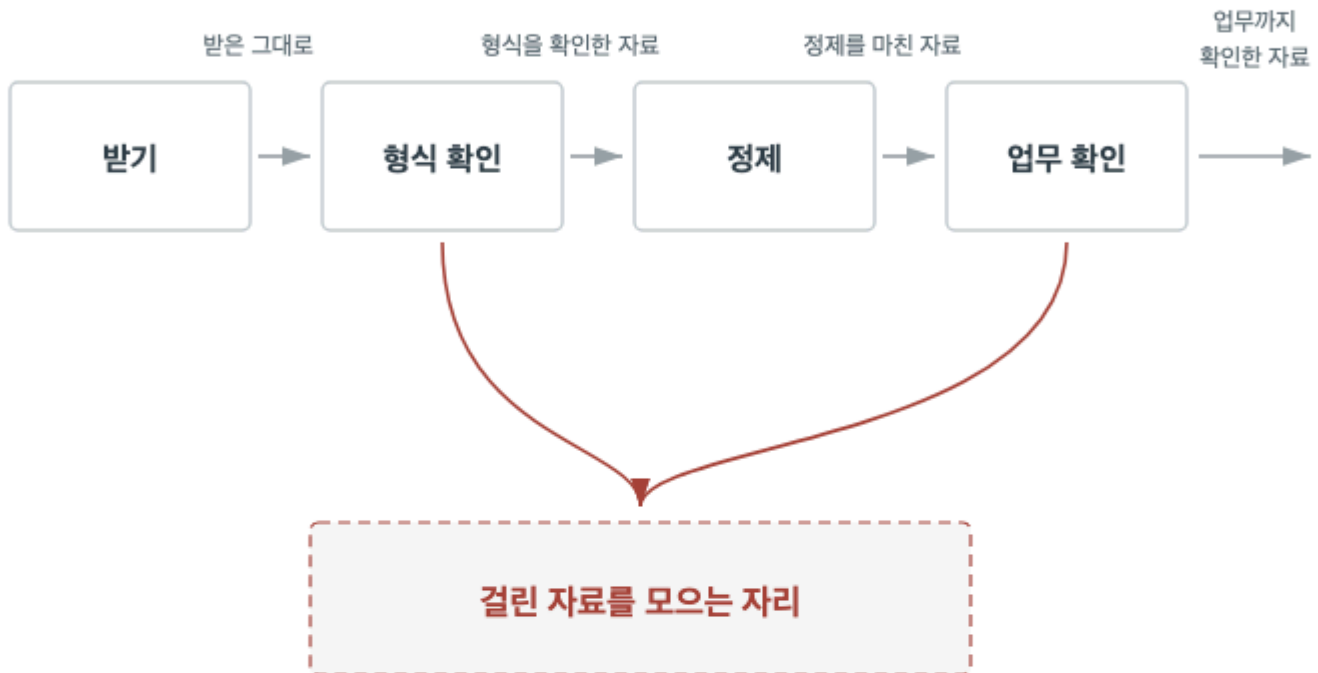
여러 자료를 한 번에 처리할 때 하나가 걸리면 선택이 갈린다. 전체를 멈출지, 걸린 것만 빼고 계속할지다. 앞쪽은 자료 사이에 관계가 있을 때 맞고, 뒤쪽은 자료가 서로 독립일 때 맞다. 이 판단을 파이프라인 전체에 하나로 정해 두면 단계마다 달라지지 않는다.

걸린 것만 빼는 방식에서는 뺀 자료를 어디에 둘지 정해야 한다. 그냥 버리면 무엇이 빠졌는지 알 수 없고, 처리한 자료에 섞으면 잘못된 값이 안으로 들어온다. 걸린 자료를 따로 모으는 자리를 두고 그 자리를 사람이 보게 하는 방식이 일반적이다.

전체를 멈추는 방식에서는 이미 처리한 부분을 어떻게 할지 정해야 한다. 앞 단계에서 저장까지 끝난 자료가 있다면 되돌리거나, 다시 돌려도 결과가 같도록 만들어 두어야 한다. 어느 쪽도 정하지 않으면 실패한 뒤의 상태가 매번 다르다.

파이프라인 단계와 자료 상태

상태의 이름이 곧 그 단계의 보장이다



걸린 자료는 버리지 않고 한자리에 모은다

5.5 다시 돌릴 수 있는 단계

파이프라인은 언젠가 중간에서 멈춘다. 그때 어디부터 다시 돌릴지 정해 두지 않으면 사람이 매번 판단해야 하고, 판단이 틀리면 자료가 두 번 들어가거나 빠진다. 다시 돌릴 수 있게 만드는 일은 예외 상황을 위한 준비가 아니라 기본 설계다.

같은 자료를 두 번 처리해도 결과가 같으려면 단계가 자기 결과를 덮어쓰는 방식이어야 한다. 이미 있으면 더하지 않고 갱신하는 방식이 대표적이다. 반대로 셀 때마다 더하는 방식은 두 번 돌리면 값이 달라진다.

자료마다 어디까지 왔는지 표시를 남기면 다시 돌릴 범위를 좁힐 수 있다. 표시가 없으면 처음부터 다시 돌려야 하고, 양이 많으면 그 자체가 부담이 된다. 상태 표시와 덮어쓰기 두 가지가 재실행을 가능하게 하는 최소 조건이다.

5.6 중간 결과를 남기는 기준

단계 사이의 자료를 남기면 다시 돌릴 때 앞 단계를 건너뛸 수 있고, 어느 단계에서 값이 달라졌는지 볼 수 있다. 대신 저장할 자리가 늘고 같은 자료의 사본이 여러 벌 생긴다. 사본이 늘면 어느 것이 기준인지 정해 두어야 한다.

남길 가치가 큰 자리는 되돌리기 비싼 단계의 앞이다. 시간이 오래 걸리는 단계나 바깥에 요청을 보내는 단계 앞에서 자료를 남겨 두면 다시 돌릴 때 그 비용을 치르지 않는다. 반대로 계산이 빠른 단계 사이의 자료는 남길 이유가 적다.

남기기로 했다면 언제 지울지도 함께 정한다. 지우는 규칙이 없으면 중간 결과가 계속 쌓여 원본보다 커지고, 나중에는 어느 것을 지워도 되는지 판단할 수 없게 된다. 남기는 결정에는 늘 지우는 규칙이 따라붙는다.

5.7 파이프라인 밖으로 나가는 자료

들어오는 자료에 스키마를 두었다면 나가는 자료에도 두어야 한다. 받는 쪽에서 보면 이 시스템이 바깥이고, 이 시스템이 보내는 자료가 신뢰할 수 없는 입력이다. 안에서 확인했다는 사실은 받는 쪽이 알 수 없다.

나가는 자료의 스키마는 안의 저장 구조를 그대로 옮기지 않는다. 그대로 내보내면 저장 구조를 고칠 때마다 받는 쪽이 함께 바뀌어야 한다. 나가는 모양은 받는 쪽이 알아야 할 것만 담아 따로 정하고, 안의 자료를 그 모양으로 옮기는 자리를 둔다.

내보내기 전에 확인할 것도 있다. 안에서 만든 자료라도 계산 결과가 약속한 범위를 벗어날 수 있고, 비어 있어서는 안 되는 항목이 빈 채로 만들어질 수 있다. 나가는 자리에서 한 번 확인하면 잘못을 받는 쪽이 아니라 만든 쪽에서 발견한다.

6.1 사례의 자료와 목적

이 장은 다른 시스템에서 하루에 한 번 받는 주문 목록 하나를 따라간다. 목록에는 주문 번호, 받는 사람, 연락처, 품목과 수량, 요청 날짜가 들어 있고, 한 번에 수천 건이 온다. 목적은 이 목록을 안의 주문 자료로 만드는 것이다.

이 자료를 고른 이유는 앞 장에서 다룬 문제가 모두 나타나기 때문이다. 표기가 통일되어 있지 않고, 빠진 항목이 있고, 안의 기록과 대조해야 판단할 수 있는 조건이 있으며, 한 건이 걸렸다고 나머지를 버릴 수 없다.

따라가는 순서는 실제 처리 순서와 같다. 받은 그대로를 보고, 형식에서 걸린 것을 보고, 정제에서 고친 것을 보고, 업무 규칙에서 걸린 것을 본 뒤, 걸린 자료를 어떻게 다시 받았는지까지 본다. 각 자리에서 무엇을 보고 무엇을 정했는지가 이 장의 내용이다.

6.2 받은 그대로의 자료

규칙을 만들기 전에 받은 자료를 먼저 보았다. 항목 이름은 약속과 같았지만 값의 모양은 한 가지가 아니었다. 연락처는 붙임표가 있는 것과 없는 것이 섞여 있었고, 날짜는 두 가지 차례가 함께 있었으며, 수량은 대부분 숫자였지만 일부는 앞뒤에 공백이 있었다.

빠진 항목도 있었다. 받는 사람이 비어 있는 건이 소수 있었고, 요청 날짜가 없는 건이 그보다 많았다. 둘 다 약속으로는 필수였다. 약속과 실체가 다르다는 사실 자체가 이 단계에서 얻은 결과다.

이 관찰을 규칙으로 바로 옮기지는 않았다. 지금 자료가 이렇다는 것과 앞으로 이래도 된다는 것은 다르기 때문이다. 관찰은 어떤 규칙이 실제로 걸릴지 짐작하는 데 쓰고, 규칙 자체는 보내는 쪽과의 약속에서 가져왔다.

6.3 형식 확인에서 걸린 것

형식 확인은 항목이 있는지, 종류가 맞는지, 값이 범위에 드는지 순서로 걸었다. 수천 건 가운데 걸린 것은 세 갈래였다. 받는 사람이 비어 있는 건, 수량이 영이거나 음수인 건, 요청 날짜가 날짜 모양이 아닌 건이다.

걸린 건은 처리에서 빼고 따로 모았다. 모을 때는 주문 번호와 어긋난 항목 이름과 어긋난 조건을 함께 적었다. 자료 전체를 그대로 옮기지 않은 것은 연락처처럼 남기지 않기로 한 값이 섞여 있기 때문이다.

한 건에서 두 항목이 함께 걸린 경우가 있어 위반을 모두 모아 적었다. 처음 하나에서 멈췄다면 보내는 쪽이 고쳐 다시 보낸 뒤 두 번째 잘못을 알게 되었을 것이다. 형식 확인 안에서는 모두 모으고, 형식에서 걸린 건은 다음 단계로 넘기지 않았다.

6.4 정제에서 고친 것

형식 확인을 지난 자료에서 세 항목을 정제했다. 연락처는 붙임표를 빼 숫자만 남겼고, 날짜는 정해 둔 하나의 차례로 옮겼으며, 수량은 앞뒤 공백을 없앴다. 셋 다 되돌릴 수 있는 변환이라 정제 범위에 두었다.

고치지 않은 항목도 있다. 비고란은 사람이 자유롭게 적는 자리라 표기를 맞춤 기준이 없었고, 맞추면 뜻이 갈릴 수 있어 받은 그대로 두었다. 품목 이름도 비슷한 이름이 여럿 있었지만 임의로 묶지 않았다. 묶는 규칙을 만들려면 보내는 쪽과의 합의가 먼저다.

고친 항목은 어떤 규칙으로 고쳤는지 건별로 남겼다. 값 자체는 남기지 않고 항목 이름과 적용한 규칙만 남겼다. 나중에 연락처 표기가 달라졌을 때 이 기록으로 어느 규칙이 언제부터 걸렸는지 확인할 수 있다.

6.5 업무 규칙에서 걸린 것

정제를 마친 자료에 업무 규칙을 걸었다. 여기서 걸린 것은 형식 확인에서 걸릴 수 없는 종류였다. 이미 처리한 주문 번호가 다시 들어온 건, 안에 없는 품목 번호를 가리키는 건, 요청 날짜가 오늘보다 앞선 건이다.

셋 다 값만 보아서는 멀쩡하다. 주문 번호는 형태가 맞고, 품목 번호도 자릿수가 맞으며, 날짜도 날짜 모양이다. 저장된 기록이나 지금 시각과 대조해야 어긋남이 드러난다. 이 확인을 들어오는 자리에 두지 않은 이유가 여기 있다.

다시 들어온 주문 번호는 실패로 다루지 않고 건너뛰었다. 보내는 쪽이 같은 목록을 두 번 보내는 일이 드물지 않고, 두 번 들어와도 결과가 같아야 다시 돌릴 수 있기 때문이다. 나머지 둘은 걸린 자료를 모으는 자리로 보냈다.

6.6 걸린 자료를 다시 받은 방법

걸린 건은 목록으로 만들어 보내는 쪽에 돌려주었다. 목록에는 주문 번호와 어긋난 항목과 어긋난 조건만 담았고, 고친 값을 제안하지는 않았다. 어떻게 고칠지는 보내는 쪽의 판단이고, 이쪽에서 제안하면 그 제안이 곧 새 규칙처럼 굳는다.

고쳐서 다시 온 건은 처음부터 같은 단계를 다시 지나게 했다. 형식은 지났으니 업무 확인만 하자는 방법도 있었지만, 그러면 재접수 경로에만 다른 규칙이 생긴다. 통로가 늘어도 규칙은 하나여야 한다는 원칙을 여기서 지켰다.

재접수한 건 가운데 다시 걸린 것이 있었다. 같은 항목이 두 번 걸린 건은 따로 표시해 사람이 보게 했다. 두 번 이상 걸리는 건은 자료의 문제가 아니라 규칙과 실제 업무가 어긋난 신호일 수 있기 때문이다.

6.7 이 길에서 남긴 기록

한 번의 처리가 끝나면 네 가지가 남는다. 처리한 건수, 단계별로 걸린 건수와 규칙, 정제된 항목과 규칙, 그리고 걸린 자료를 모은 자리의 내용이다. 이 넷이 다음 판단의 재료가 된다.

며칠 치를 견주자 규칙마다 걸리는 빈도가 크게 달랐다. 요청 날짜가 비어 오는 건이 꾸준히 많았는데, 확인해 보니 보내는 쪽에서 그 항목을 선택으로 다루고 있었다. 약속은 필수였지만 실제 운영은 달랐던 것이다.

이 어긋남은 자료를 고쳐 해결할 일이 아니라 약속을 고칠 일이었다. 기록이 없었다면 매일 같은 건수가 걸리는 것을 처리 실패로만 보았을 것이다. 검증 기록의 쓸모는 막은 자료를 세는 데 있지 않고 규칙이 실제와 맞는지 보는 데 있다.

7.1 과도한 검증의 비용

검증을 늘리면 안전해진다는 말은 어느 지점까지만 맞다. 규칙이 늘수록 정상 자료가 막히는 일도 함께 늘고, 막힌 자료를 사람이 확인하는 일이 늘며, 규칙을 고칠 때 확인할 자리도 늘어난다.

특히 실제 업무에서 오는 예외를 규칙으로 막으면 비용이 크다. 드물지만 정상인 자료가 매번 걸리면 사람은 결국 그 규칙을 우회할 방법을 찾는다. 우회로가 생기면 검증은 형식만 남고 실제로는 동작하지 않는다.

규칙을 더할지 판단할 때는 두 가지를 본다. 이 규칙이 막으려는 문제가 실제로 일어났는가, 그리고 막지 못했을 때의 손해가 잘못 막았을 때의 손해보다 큰가. 둘 다 그렇지 않으면 규칙 대신 기록을 남기고 지켜보는 편이 낫다.

7.2 어디까지 막을 것인가

모든 이상한 값을 막을 필요는 없다. 막아야 하는 것은 안으로 들어오면 되돌리기 어려운 값이다. 저장되어 다른 계산에 쓰이거나, 바깥으로 다시 나가거나, 사람의 판단 근거가 되는 값이 여기 든다.

확인하지 않은 값은 들어온 자리에서 멈추지 않고 사본으로 퍼져 원인을 가리기 어렵게 만든다. 그래서 되돌리기 어려운 자리 앞에 확인선을 둔다. 반대로 들어와도 곧 사라지는 값, 사람이 곧바로 다시 보는 값은 막기보다 기록하고 지켜보는 편이 낫다.

이 기준으로 보면 검증 설계는 규칙을 모으는 일이 아니라 되돌리기 어려운 자리를 찾는 일이다. 자리를 찾으면 규칙은 따라오고, 자리를 찾지 못한 채 규칙만 늘리면 어디를 지키고 있는지 알 수 없는 검증이 남는다.

7.3 스키마를 넓힐 때와 좁힐 때

스키마를 넓히는 것은 더 많은 자료를 받아들이는 일이다. 선택 항목을 더하거나 값의 범위를 늘리면 보내는 쪽은 편해지지만 받는 쪽이 다뤄야 할 경우가 늘어난다. 넓히기 전에 그 경우를 안에서 실제로 처리할 수 있는지 확인한다.

좁히는 것은 이미 받아들이던 자료를 막는 일이라 더 조심스럽다. 좁히기 전에 지금까지 들어온 자료 가운데 얼마나 막히는지 세어 보아야 하고, 막히는 자료가 있다면 보내는 쪽과 시점을 합의해야 한다. 검증 기록이 있으면 이 계산을 짐작이 아니라 수로 할 수 있다.

어느 방향이든 한 번에 바꾸지 않는다. 새 규칙을 걸되 처음에는 막지 않고 기록만 남겨 얼마나 걸리는지 보고, 그다음에 막는다. 이 순서를 지키면 규칙 하나가 처리 전체를 멈추는 일이 생기지 않는다.

7.4 검증 설계 점검 항목

검증 설계를 점검할 때는 다음을 차례로 본다. 첫째, 자료가 들어오는 통로를 모두 적었는가. 적지 않은 통로가 있으면 그 통로는 확인선 밖이다. 둘째, 같은 조건을 여러 자리에서 확인하고 있는가. 겹친 확인은 규칙이 갈라지는 자리다.

셋째, 형식 확인과 업무 확인이 나뉘어 있는가. 섞여 있으면 형식 하나를 고칠 때도 업무 상태를 갖춰야 한다. 넷째, 막힌 자료가 어디로 가는가. 버려지고 있다면 무엇이 빠졌는지 아무도 모른다.

다섯째, 어떤 규칙에 얼마나 걸리는지 셀 수 있는가. 셀 수 없으면 규칙을 고칠 때마다 짐작으로 판단하게 된다. 이 다섯은 자료를 한 건도 흘려보내지 않고 설계 문서와 코드만 읽어 확인할 수 있으며, 걸리는 항목마다 앞 장에 대응하는 절이 있다.

7.5 정제 규칙을 다시 볼 때

정제 규칙은 한 번 정하면 오래 남는다. 자료의 모양이 바뀌어도 규칙은 그대로 돌아가고, 규칙이 실제와 어긋나도 오류가 나지 않는다. 정제는 막는 일이 아니라 고치는 일이라 어긋나도 조용하다.

그래서 정제 규칙은 정기적으로 다시 본다. 볼 때 확인할 것은 두 가지다. 이 규칙이 지금도 걸리고 있는가, 그리고 걸릴 때 고치는 방향이 지금도 맞는가. 한 번도 걸리지 않는 규칙은 지워도 되고, 너무 자주 걸리는 규칙은 보내는 쪽의 표기가 바뀐 신호다.

규칙을 지울 때는 지운 뒤 어떤 자료가 어떻게 달라지는지 먼저 본다. 정제는 뒤의 처리가 그 결과를 전제하고 있으므로, 규칙 하나를 지우면 표기가 통일되지 않은 자료가 안쪽으로 들어간다. 정제 규칙의 변경은 늘 뒤 단계와 함께 본다.

7.6 파이프라인을 바꾸는 순서

돌아가고 있는 파이프라인을 고칠 때는 단계를 한 번에 하나만 손댄다. 여러 단계를 함께 바꾸면 결과가 달라졌을 때 어느 변경 때문인지 가릴 수 없다. 단계 사이의 계약이 적혀 있으면 한 단계만 바뀌어도 앞뒤가 흔들리지 않는다.

새 규칙이나 새 단계를 넣을 때는 기록만 남기는 상태로 먼저 넣는다. 얼마나 걸리는지, 어떤 자료가 걸리는지 보고 나서 실제로 막거나 고치도록 바꾼다. 처음부터 막으면 예상하지 못한 정상 자료가 함께 막힌다.

바꾼 뒤에는 같은 자료를 다시 돌려 결과를 건준다. 재실행이 가능하게 만들어 두었다면 이 확인이 쉽고, 그렇지 않다면 바꿀 때마다 새 자료를 기다려야 한다. 재실행 가능성은 고칠 수 있는 상태를 만드는 조건이기도 하다.

7.7 핵심 원칙 정리

이 책이 반복한 물음은 둘이다. 이 값은 어디서 왔는가, 그리고 이 값이 조건을 만족한다고 누가 책임지는가. 첫 물음이 신뢰 경계를 정하고, 둘째 물음이 검증할 자리를 정한다.

나머지는 이 둘에서 따라 나온다. 형식과 뜻을 나눈 것은 확인에 필요한 것이 다르기 때문이고, 정제와 검증을 나눈 것은 고치는 일과 막는 일이 다르기 때문이며, 파이프라인을 단계로 나눈 것은 자료의 상태를 말할 수 있게 하기 위해서다.

마지막으로 남길 것은 검증이 자료를 완전하게 만들지 못한다는 점이다. 검증은 어긋난 자료가 들어오는 범위를 좁히고 들어왔을 때 찾을 수 있게 만들 뿐이다. 완전함을 목표로 삼으면 규칙이 끝없이 늘고, 범위를 좁히는 것을 목표로 삼으면 어디까지 할지 정할 수 있다.